

# Model Improvements (V3)

Network Monitoring & Predictive Failure System

upgraded your project from a:

```
basic failure detection model
```

to a:

```
real-time predictive network failure analysis system
```

The improvements were not just “code changes.”

They fixed major ML architecture issues and made the model behave more like a realistic predictive monitoring system.

---

## 1. Fixed Feature Leakage Problem

### Old Problem

The model previously used:

```
fails  
prev_state_encoded
```

These features leaked near-failure information.

Example:

```
Current state = DEGRADED  
→ model easily predicts DOWN
```

This is not real prediction.

The model was basically:

```
detecting already broken nodes
```

instead of:

```
predicting future instability
```

---

## What Changed

Removed:

```
prev_state_encoded
```

Reduced dependency on:

```
fails
```

and shifted focus toward:

- latency trends
- instability
- packet loss behavior
- rolling patterns

---

## Why Necessary

Without this fix:

- accuracy becomes artificially high
- predictions are unrealistic
- model memorizes labels instead of learning patterns

After fix:

```
model learns behavior evolution
```

which is true predictive ML.

---

## 2. Increased Window Size

### Old

```
WINDOW=5
```

Meaning:

```
only ~25 seconds of history
```

### New

```
WINDOW=10
```

Now model sees:

```
~50 seconds of network behavior
```

## Improvement

This allows model to detect:

- gradual latency rise
- instability patterns

- packet-loss trends
- degradation evolution

instead of reacting only to sudden failures.

---

## 3. Improved Prediction Horizon

### Old

```
LOOKAHEAD=3
```

Too short.

---

### New

```
LOOKAHEAD=6
```

Meaning:

```
predict failure ~30 seconds ahead
```

---

## Why Important

This transformed system from:

```
current-state detector
```

to:

```
early warning system
```

Now administrators can:

- react earlier
  - stabilize nodes
  - prevent downtime
- 

## 4. Realistic Failure Simulation

### Old Behavior

Previous dataset generation:

```
healthy  
→ instantly DOWN
```

This is unrealistic.

---

### New Behavior

Failure progression now becomes:

```
healthy  
→ unstable  
→ latency increase  
→ packet loss  
→ degraded  
→ severe degradation  
→ down
```

---

### Why Important

This teaches the model:

```
how failures evolve over time
```

instead of memorizing random DOWN events.

This is one of the biggest improvements.

---

## 5. Added Ping Failure Detection

### New Feature

```
ping_failed
```

---

### Why Important

Now model distinguishes:

| Condition       | Meaning              |
|-----------------|----------------------|
| High latency    | unstable network     |
| ping_failed = 1 | connectivity failure |

This greatly improves:

- reliability
  - packet-loss understanding
  - failure prediction quality
- 

## 6. Added Rolling Packet Loss Analysis

### New Feature

```
ping_failed_rate
```

This calculates:

recent packet-loss frequency

## Improvement

Model can now identify:

repeated intermittent failures

which are strong indicators of future outages.

## 7. Added Latency Acceleration

### New Feature

latency\_acceleration

### What It Measures

Not just:

latency increasing

but:

how fast latency is worsening

### Why Important

Rapid latency acceleration is often:

```
early symptom of network collapse
```

This gives earlier prediction capability.

---

## 8. Added Latency Volatility

### New Feature

```
latency_volatility
```

---

### Purpose

Measures:

```
unstable oscillating latency
```

Example:

```
10ms → 80ms → 20ms → 150ms
```

This unstable behavior commonly appears before:

- router overload
  - packet congestion
  - device failure
- 

## 9. Improved Instability Detection

### Old

Only based on:

```
rolling_std_latency
```

---

## New

Combines:

- latency jitter
- packet-loss frequency

```
rolling_std_latency  
ping_failed_rate
```

---

## Result

Much more realistic instability detection.

---

# 10. Fixed Threshold Logic

## Old Problem

Threshold was incorrectly applied on:

```
predict()
```

instead of:

```
predict_proba()
```

---

## Fix

Now:

```
y_prob=model.predict_proba(...)
```

then threshold applied properly.

---

## Why Important

Now probabilities are:

- meaningful
  - stable
  - usable in GUI alerts
- 

# 11. Improved Random Forest Training

## Old

```
n_estimators=200  
max_depth=12
```

---

## New

```
n_estimators=300  
max_depth=15  
min_samples_split=8  
min_samples_leaf=3
```

---

## Improvement

This reduces:

- overfitting
- noise memorization
- unstable trees

while improving:

- generalization
- stability
- predictive confidence

---

## 12. Added Feature Validation

Before prediction:

```
missing_features= [...]
```

---

### Why Important

Prevents:

- silent failures
- wrong feature order
- corrupted predictions

Very important production-level improvement.

---

## 13. Added Risk Classification

Now probabilities map to:

| Probability | Risk   |
|-------------|--------|
| < 0.4       | LOW    |
| 0.4–0.6     | MEDIUM |

| Probability | Risk     |
|-------------|----------|
| 0.6–0.8     | HIGH     |
| > 0.8       | CRITICAL |

## Improvement

GUI can now display:

- intelligent alert levels
- better monitoring visibility
- easier admin interpretation

## 14. Added Model Compatibility Layer

Added:

```
predict_node_failure()
```

## Why Important

Your old GUI and monitor system expected:

```
predict_node_failure()
```

Without wrapper:

```
ML predictor import fails
```

The wrapper preserved:

- backward compatibility

- GUI integration
- monitoring pipeline

without rewriting architecture.

## 15. Full Pipeline Alignment

Now aligned:

```
Training
↔ predictor.py
↔ monitor.py
↔ GUI
↔ MongoDB logs
```

## Comparison Table — Earlier vs Improved System

| Feature / Component   | Earlier System (V1/V2)    | Improved System (V3)          | Improvement Achieved               |
|-----------------------|---------------------------|-------------------------------|------------------------------------|
| Prediction Type       | Current failure detection | Early failure prediction      | More proactive monitoring          |
| Monitoring Logic      | Reactive                  | Predictive + behavior-based   | Detects instability before failure |
| Window Size           | 5 logs (~25 sec)          | 10 logs (~50 sec)             | Better trend analysis              |
| Prediction Horizon    | 3 future rows             | 6 future rows (~30 sec ahead) | Earlier warning system             |
| Failure Simulation    | Sudden DOWN states        | Gradual failure evolution     | Realistic learning behavior        |
| Packet Loss Detection | Not included              | Added ping_failed             | Better connectivity analysis       |

|                             |                           |                                   |                                      |
|-----------------------------|---------------------------|-----------------------------------|--------------------------------------|
| Packet Loss Trend           | Not available             | Added ping_failed_rate            | Detects repeated failures            |
| Latency Trend               | Basic                     | Advanced temporal trend analysis  | Better predictive capability         |
| Latency Acceleration        | Not available             | Added latency_acceleration        | Detects rapid network degradation    |
| Latency Volatility          | Not available             | Added latency_volatility          | Captures unstable oscillation        |
| Instability Detection       | Std deviation only        | Std deviation + packet loss       | More realistic instability detection |
| Feature Leakage             | Present                   | Removed                           | Real predictive learning             |
| Threshold Logic             | Incorrect implementation  | Probability-based thresholding    | Stable predictions                   |
| Random Forest Configuration | Basic tuning              | Optimized hyperparameters         | Better generalization                |
| GUI Integration             | Basic probability display | Risk-level based alerting         | Improved visualization               |
| Alert Levels                | Simple                    | LOW / MEDIUM / HIGH / CRITICAL    | Better network administration        |
| Prediction Stability        | Noisy                     | Smoothed probabilities            | Reduced false alerts                 |
| Data Validation             | Minimal                   | Added feature validation          | More reliable inference              |
| System Architecture         | Monitoring-focused        | Intelligent predictive monitoring | More industry-like system            |